

LeakScope

Security Assessment Report

TARGET SYSTEM

<https://mathiit.live/attempt/cpFEUaz9>

ASSESSMENT DATE

August 22, 2026

ASSESSMENT TOOL

LeakScope Automated Security Auditor

OVERALL SECURITY RATING

B (SECURE Risk)

SECURE

Grade: B

1 MEDIUM

2 LOW

Executive Summary

3 actionable finding(s) were analyzed with a local fallback due to AI response formatting limits. Current scan score is 86/100.

SEVERITY DISTRIBUTION

Severity	Findings
Critical	0
High	0
Medium	1
Low	2
Informational	1

KEY RISK AREAS

- Potential data modification risks

Scope and Methodology

The objective of this assessment was to identify potential security weaknesses in the target system.

The scan was conducted using the LeakScope automated security analysis framework, which evaluates application infrastructure for:

- Misconfigured database permissions
- Weak access control policies
- Exposed data resources
- Potential privilege escalation vectors
- Authentication and token security
- Storage and file access controls

The scan simulated anonymous user interaction, allowing the system to identify vulnerabilities that may be exploitable without authentication.

TOOLS USED

- LeakScope Automated Security Auditor

Risk Overview

The assessment did not identify any critical or high-severity vulnerabilities. The current security posture appears reasonable, though continued monitoring and periodic assessments are recommended.

Scan Details

Target URL	https://mathiit.live/attempt/cpFEUaz9
Scan Date	August 22, 2026
Scan Type	Full automated scan
Authentication	Anonymous
Bundles Scanned	20
Tables Discovered	3
Total Findings	4

SEVERITY BREAKDOWN

Severity	Findings
Critical	0
High	0
Medium	1
Low	2
Informational	1

SECURITY SCORE

MEDIUM	1 findings x 8 pts = -8
LOW	2 findings x 3 pts = -6

Overall Score: 86 / 100

Grade: B

Findings

#01

CVSS: 5.5

MEDIUM

Missing Content-Security-Policy Header

DESCRIPTION

No Content-Security-Policy header is set. Without a CSP, the browser has no directive restrictions and any injected inline script or external script source will execute freely — the primary enabler of Cross-Site Scripting (XSS) attacks.

TECHNICAL EVIDENCE

Header Name: Content-Security-Policy

Header Value: (absent)

Target Url: <https://mathiit.live/attempt/cpFEUaz9>

REMEDIATION CHECKLIST

- ☐ Restrict access with least-privilege policy and ownership checks.
- ☐ Re-run the same scan path to confirm the finding is no longer reproducible.

IMPLEMENTATION DETAILS

[illegible]

#02

CVSS: 3.1

LOW

Database Tables Discoverable via Name Fuzzing

DESCRIPTION

The PostgREST schema endpoint is restricted, but common table names were confirmed to exist by probing /rest/v1/<tableName>. Table names leak information about data architecture.

TECHNICAL EVIDENCE

Discovered Tables: ["leads","students","courses"]

Method: HEAD probe against common table name list

REMEDIATION CHECKLIST

- ☐ Review and restrict access controls for the affected resource
- ☐ Implement authentication and authorization checks
- ☐ Conduct a follow-up security review after remediation

IMPLEMENTATION DETAILS

This is expected PostgREST behavior. Ensure RLS is enabled on ALL discovered tables.

#03

CVSS: 3.5

LOW

Missing Clickjacking Protection (X-Frame-Options)

DESCRIPTION

No X-Frame-Options header is set and the CSP does not include frame-ancestors. This allows any site to embed your app in an <iframe>, enabling clickjacking attacks where users are tricked into clicking hidden actions (confirm payments, change settings, etc.).

TECHNICAL EVIDENCE

Header Name: X-Frame-Options

Header Value: (absent)

Cs Frame Ancestors: (absent)

Target Url: <https://mathiit.live/attempt/cpFEUaz9>

REMEDIATION CHECKLIST

- ☐ Review and restrict access controls for the affected resource
- ☐ Implement authentication and authorization checks
- ☐ Conduct a follow-up security review after remediation

IMPLEMENTATION DETAILS

```
# % % Nginx % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
add_header X-Frame-Options "DENY" always;  
  
# Or via CSP (preferred – overrides X-Frame-Options in modern browsers):  
add_header Content-Security-Policy "frame-ancestors 'none'" always;  
  
# % % Express.js (helmet) % % % % % % % % % % % % % % % % % % % % % % % % % %  
app.use(helmet.frameguard({ action: "deny" }));  
  
// Or via CSP:
```

```
app.use(
  helmet({
    contentSecurityPolicy: {
      directives: {
        frameAncestors: ["'none'"],
      },
    },
  })
);
```

#04

CVSS: 0.0

INFO

No unauthenticated data access detected

DESCRIPTION

Probed all common table names without any API key. No data was returned without credentials. Paste your app URL for a full scan including anon-key-level RLS and BOLA tests.

TECHNICAL EVIDENCE

Tables Probed: 65

Credentials Used: none

REMEDIATION CHECKLIST

- ☐ Review and restrict access controls for the affected resource
- ☐ Implement authentication and authorization checks
- ☐ Conduct a follow-up security review after remediation

IMPLEMENTATION DETAILS

Paste your app URL (e.g. <https://yourapp.vercel.app>) for a full scan.

Remediation Roadmap

To reduce risk efficiently, remediation efforts should follow a structured priority model.

Priority 3 — This Month (Medium & Low Severity)

Address during normal maintenance cycle.

- Strengthen configuration defaults
- Improve monitoring and logging mechanisms
- Conduct periodic security audits

Affected findings (3):

- ☐ Missing Content-Security-Policy Header
- ☐ Database Tables Discoverable via Name Fuzzing
- ☐ Missing Clickjacking Protection (X-Frame-Options)

SLA RECOMMENDATIONS

Severity	Recommended Fix Time
Critical	7 Days
High	30 Days
Medium	60–90 Days
Low	As Scheduled

Conclusion

The security assessment identified 4 findings, none of which are classified as critical or high severity.

While the current security posture is reasonable, addressing the remaining medium and low severity items will further strengthen the application's defenses.

A follow-up security assessment is recommended after remediation to confirm all findings have been resolved.